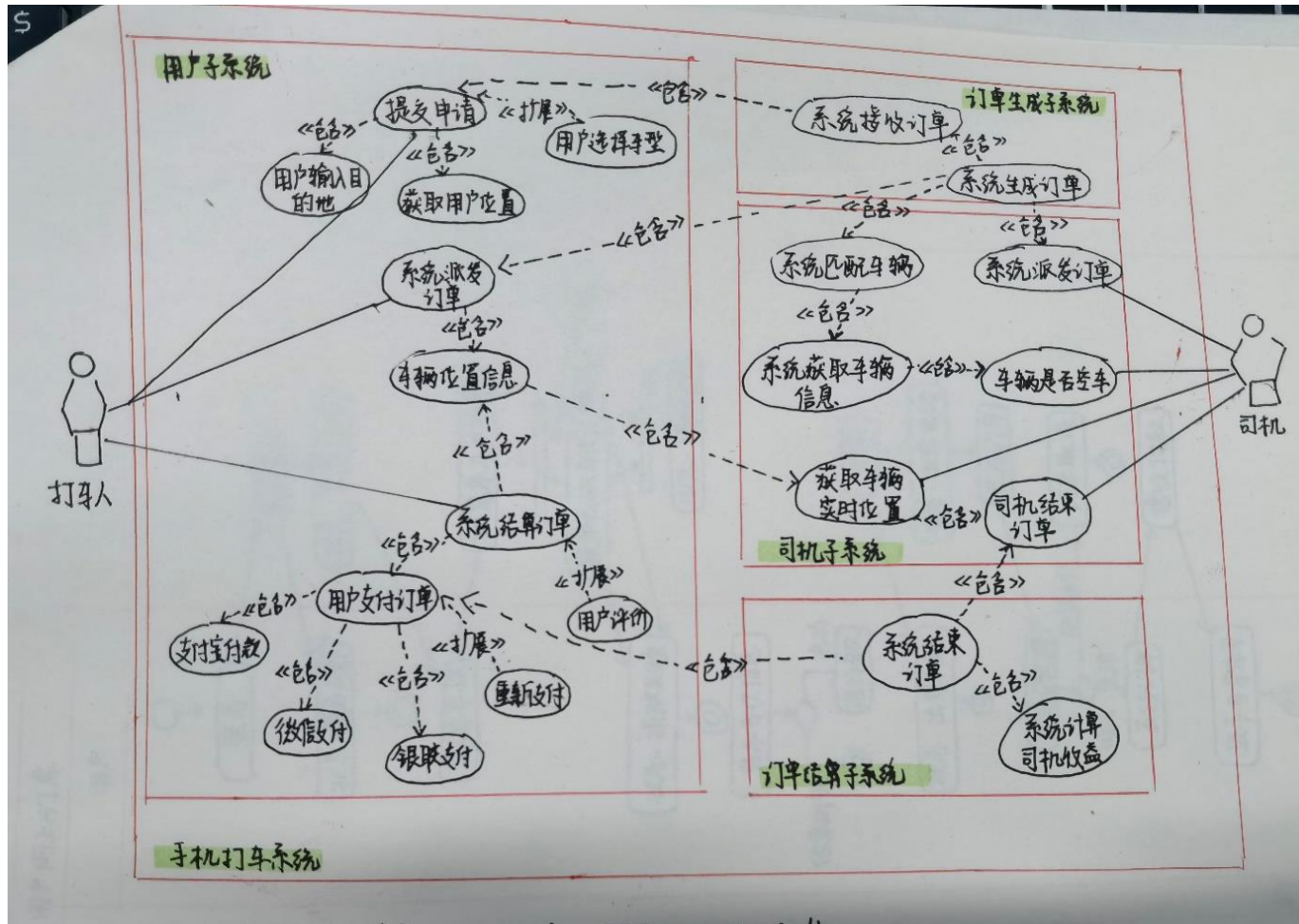


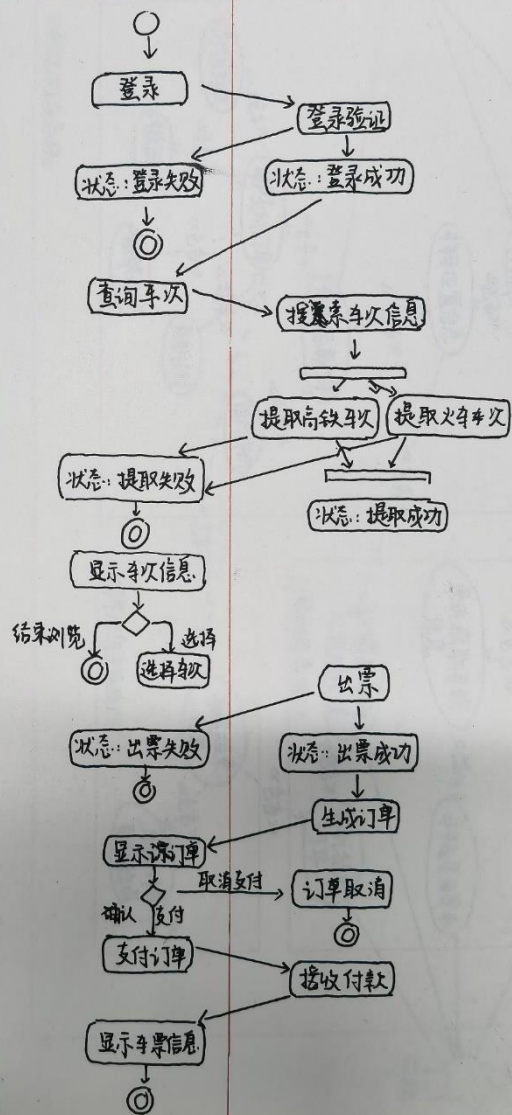
第七章 软件详细设计



用户网上订票

用户

订票软件/网站

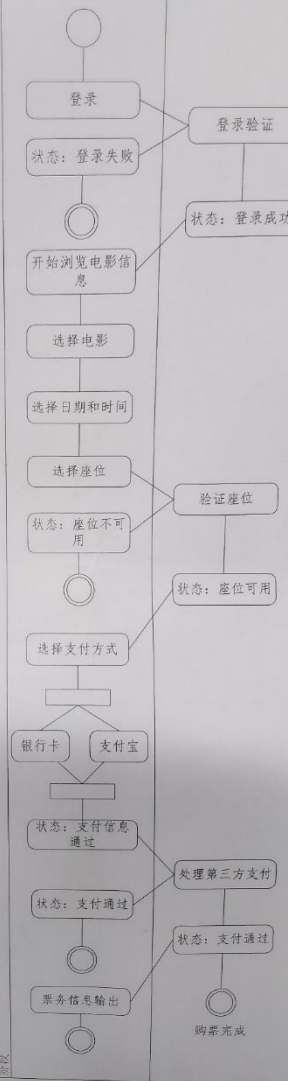


阶段

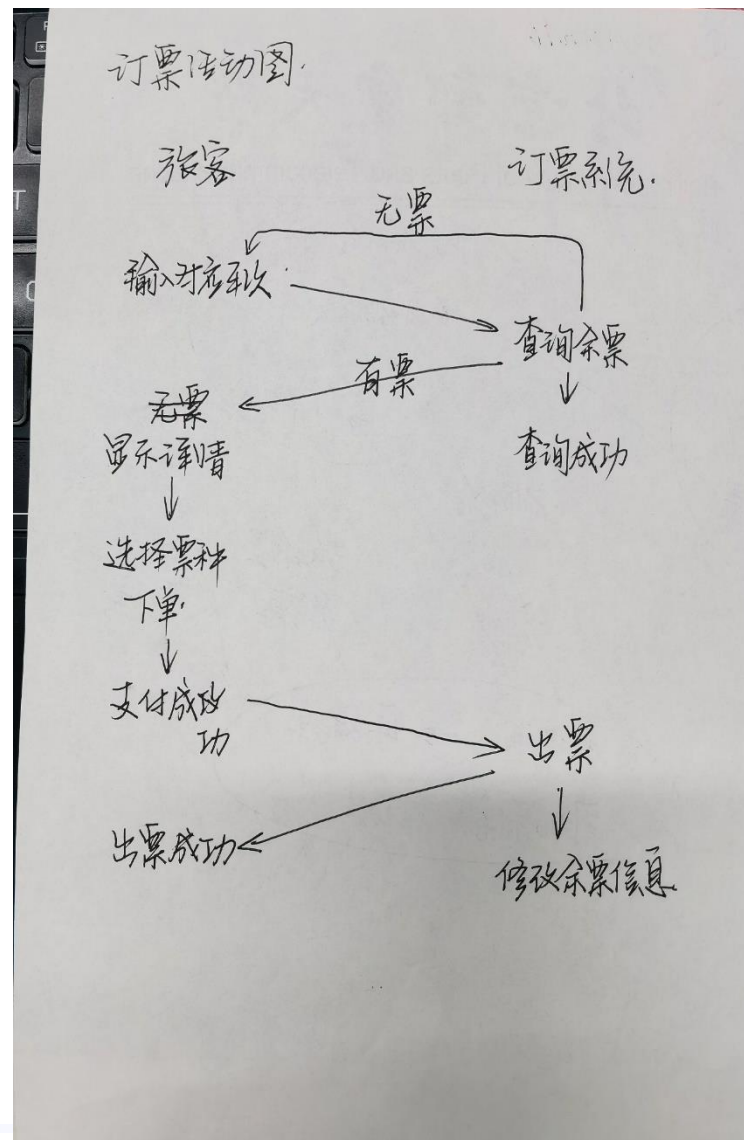
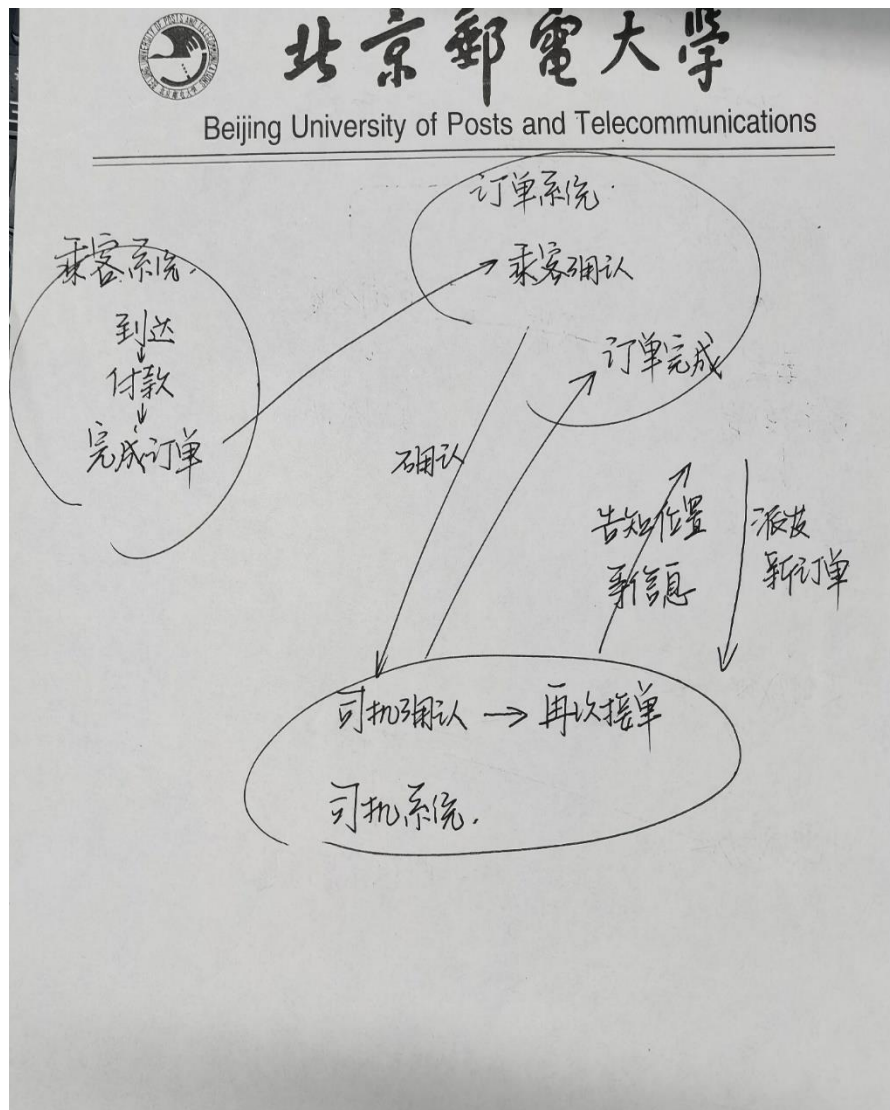
网上订购电影票

用户

网站

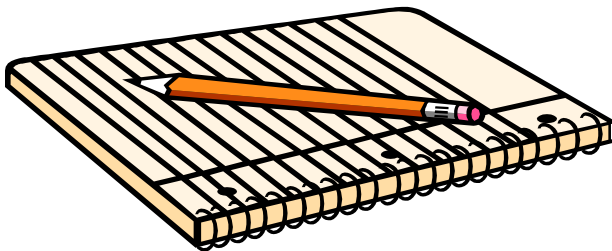


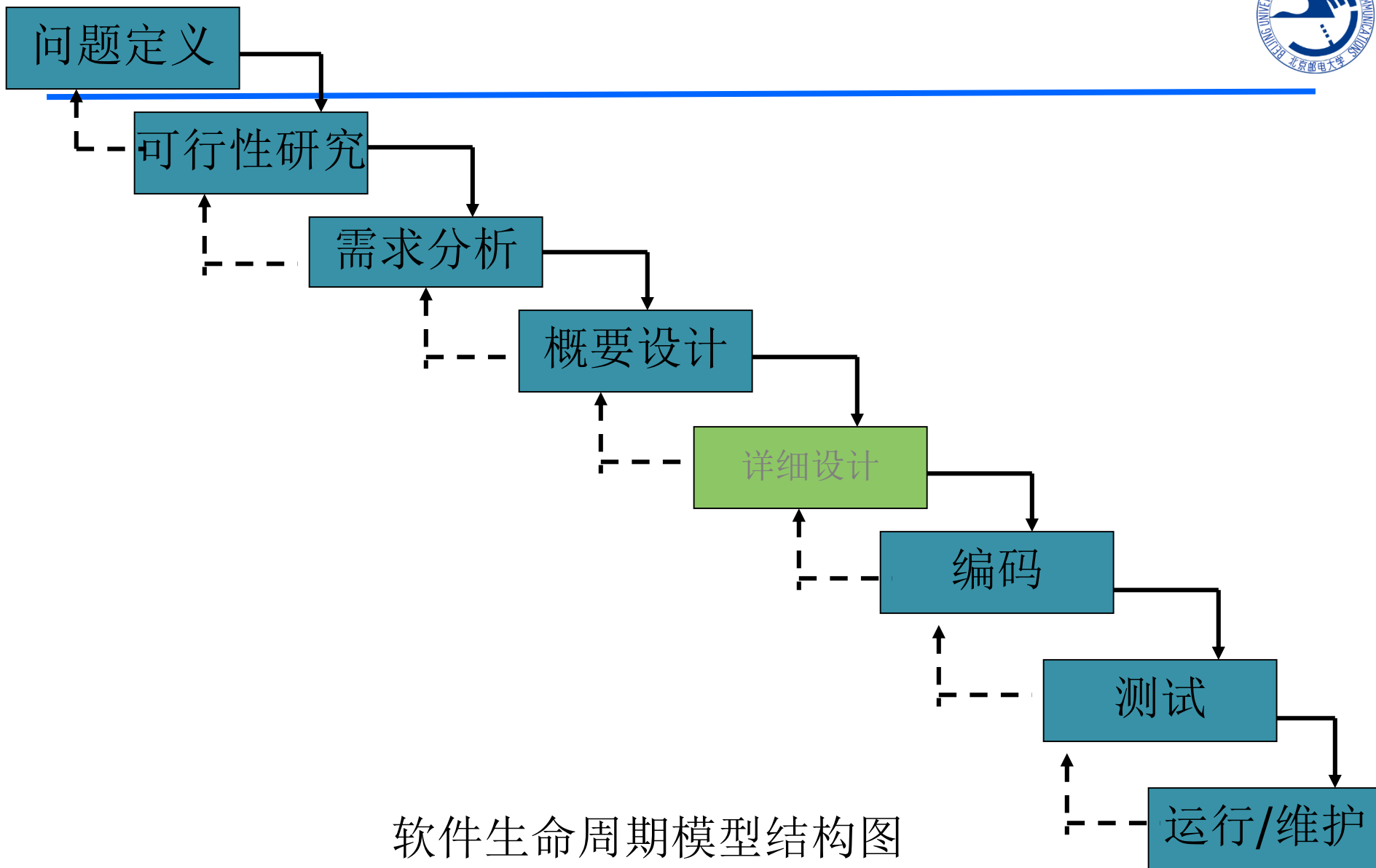
阶段



本章要点

- 详细设计的概念
- 结构程序设计
- 过程设计工具

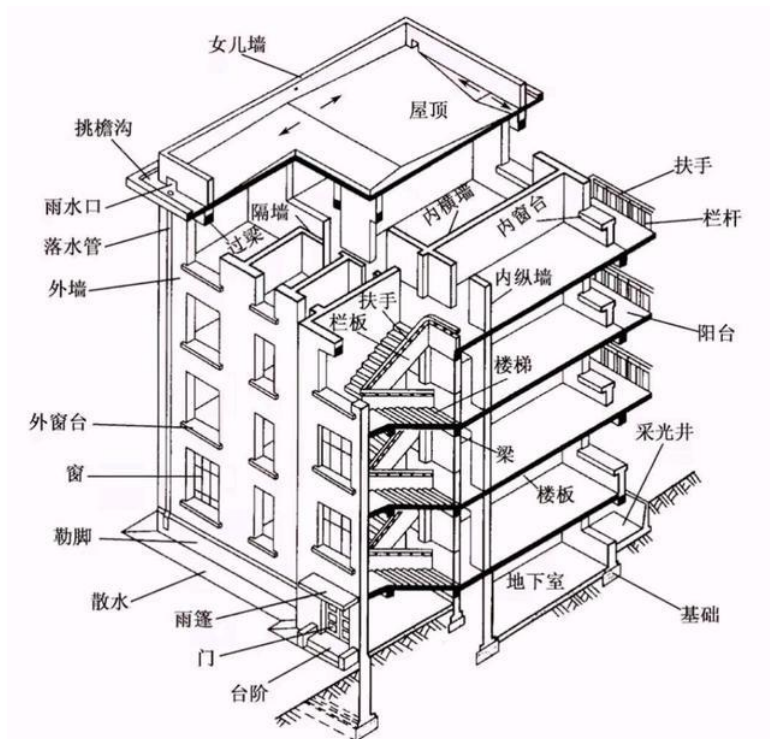




7.1 详细设计的概念

- 详细设计也叫做过程设计或程序设计，它不同于编码或编程。在过程设计阶段，要决定各个模块的实现算法，并精确地表达这些算法，即设计出程序的详细规格说明（数据结构+算法）
- 从软件开发的工程化观点来看，在使用程序设计语言编制程序以前，需要对所采用算法的逻辑关系进行分析，设计出全部必要的过程细节，并给予清晰的表达，使之成为编码的依据，这就是详细设计的任务。
- Program Design $\neq$ Coding

概要设计与详细设计



详细设计的优化

- 在不考虑时间因素的前提下开发并精化软件结构
- 选出最耗时的模块，仔细设计处理算法
- 孤立出大量占有处理机资源的模块
- 必要时重新设计或用依赖于机器的语言重写大量占有资源的模块

详细设计的过程

1. 确定每个模块的算法，选择适当工具表达算法执行过程；
2. 确定每一个模块的数据结构；
3. 为每一个模块设计一组测试用例；
4. 编写《详细设计说明书》；
5. 设计评审。

详细设计说明书的内容

- 程序描述：程序简要描述，意义和特点
- 功能：程序应具备的功能
- 性能：精度、灵活性和时间特性等
- 输入项
- 输出项
- 算法：具体的计算步骤和过程
- 接口：模块的隶属关系、调用方式和参数、与模块直接关联的数据结构（如数据库表、文件等）
- 存储分配（根据需要）
- 注释设计：程序注释说明
- 限制条件：程序运行中所受的限制条件

有些人觉得详细设计是否有必要？

- 当详细设计和编码人员实际为同一人时

观点：既然是同一人，自己做事心里有数，整体想清楚了就开始编码，而不愿意写详细设计文档

- 当详细设计和编码人员不是同一人时：

观点：详细设计要画的很细才有效果，如果详细设计做的不够细，那还不如直接看需求文档

- 对此你怎么看？

7.2 结构化程序设计

结构化程序设计的提出

- 结构程序设计概念最早1965年由E. W. Dijkstra提出

“可以从高级语言中取消GOTO语句”

“程序的质量与程序中所包含的GOTO语句的数量成反比”

- 1966年bohm和Jacopini证明了

只用三种基本的控制结构能实现任何单入口单出口的程序

结构化程序设计

- 1971年IBM公司成功地使用了结构程序设计技术

纽约时报信息库管理系统

美国宇航局空间实验室飞行模拟系统

- 1972年IBM公司的Mills进一步补充了结构程序设计的规则

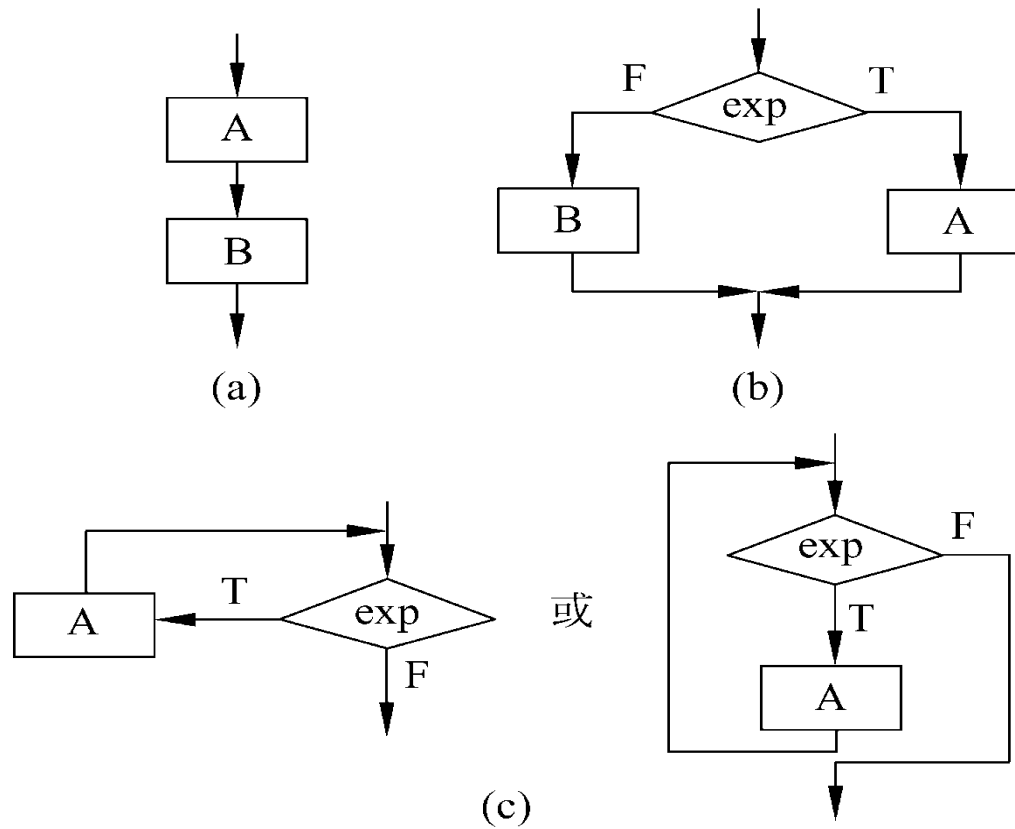
结构程序设计是尽可能少用GOTO语句的程序设计方法。最好仅在检测出错误时才使用GOTO语句，而且应该总是使用前向GOTO语句

结构程序

- 只允许使用顺序、IF-THEN-ELSE分支和DO-WHILE型循环这三种基本控制结构，称为经典的结构程序设计
- 还允许使用DO-CASE多分支结构和DO-UNTIL循环结构，称为扩展的结构程序设计
- 如果再加上允许使用LEAVE(或BREAK)结构，称为修正的结构程序设计

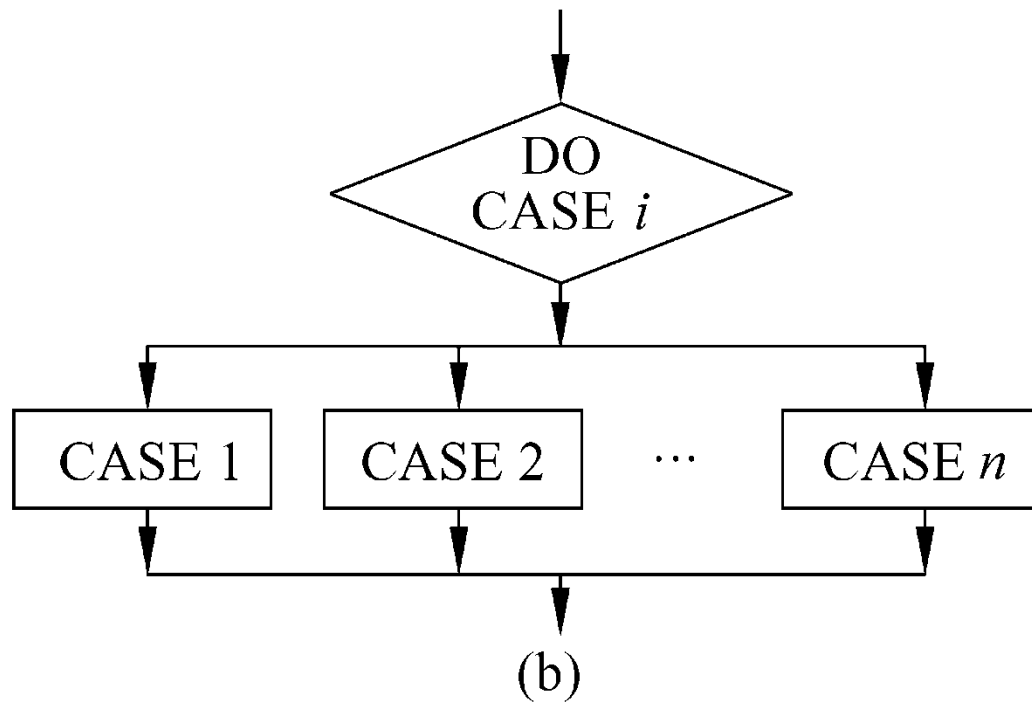
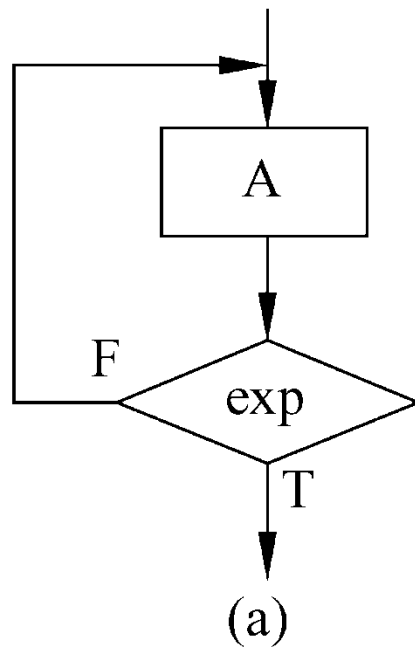
结构程序设计结构

- 3种基本的控制结构



结构程序设计结构

- 扩展的结构化程序设计



结构程序设计遵循的基本原则

- 1. 采用自顶向下、逐步求精的模块化方法设计程序

- 自底向上的程序设计方法，编写出的程序往往局部结构较好，而整体结构不佳，有时会导致程序设计不理想或失败。
- 自顶向下逐步求精的方法符合人们解决复杂问题的普遍规律，整体结构清晰、合理。
- 软件可读性好，提高了软件的可维护性

结构程序设计遵循的基本原则

- 2. 尽量使用“基本结构”编程

- 3. 限制或不使用goto语句

- 大量的资料数据表明：软件产品的质量与软件中goto语句的数量成反比；

结构程序设计的好处

- 自顶向下逐步求精的软件开发方法比较符合人们解决复杂问题的规律，因此可以提高软件开发的成功率和生产率。
- 由全局到局部，由整体到细节的逐步求精过程开发出的软件具有良好的层次结构，更易于开发人员的阅读和理解。
- 限制或不使用goto语句，使得程序的静态结构和动态执行情况较为接近，对于错误的诊断和纠正比较容易。

7.3 详细设计工具

- 在详细设计阶段，要决定各个模块的实现算法，并精确地表达这些算法。表达过程规格说明的工具叫做详细设计工具，它可以分为以下三类：

图形工具

表格工具

语言工具

过程设计工具

- 图形工具

 - 程序流程图 (FC)

 - 盒图 (NS)

 - 问题分析图 (PAD)

- 表格工具

 - 判定表

 - 判定树

- 语言工具

 - 过程设计语言 (PDL)

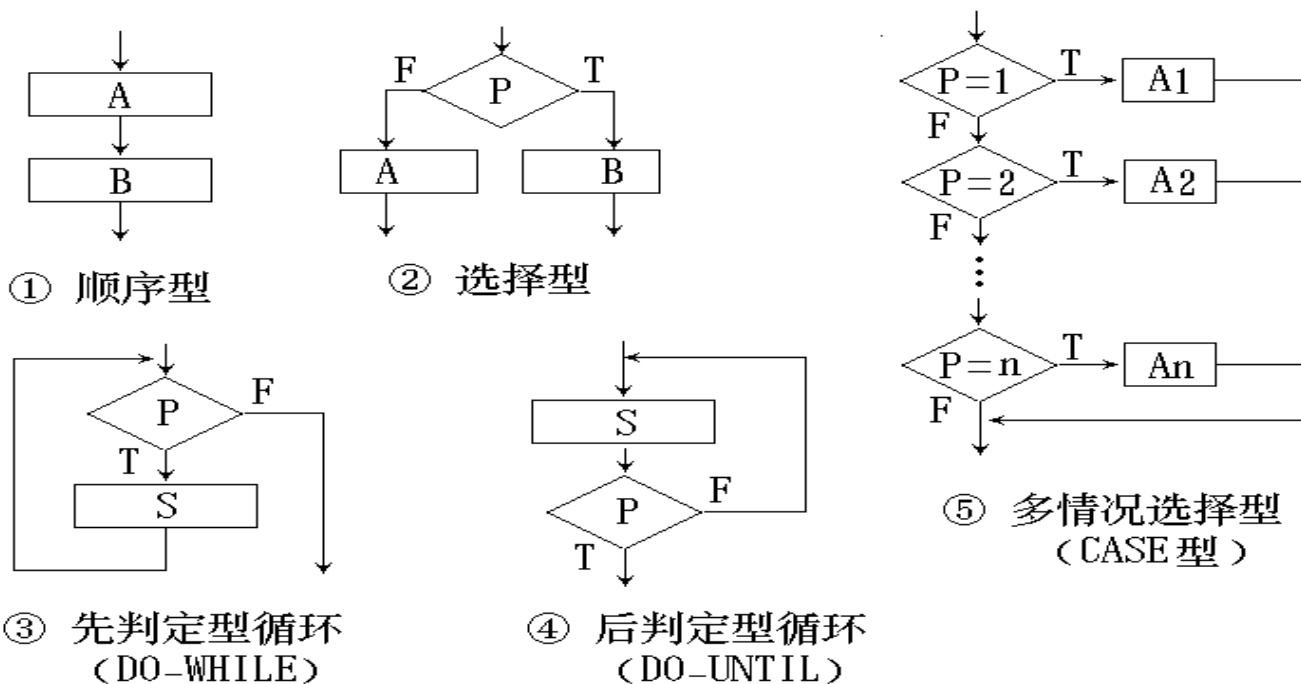
7.3.1 过程设计工具--程序流程图

1. 程序流程图

- 程序流程图独立于任何一种程序设计语言，比较直观、清晰，易于学习掌握。
- 在40年代末到70年代中期，程序流程图一直是软件设计的工具
- 它以对控制流程的描绘直观、易于掌握而被设计人员青睐

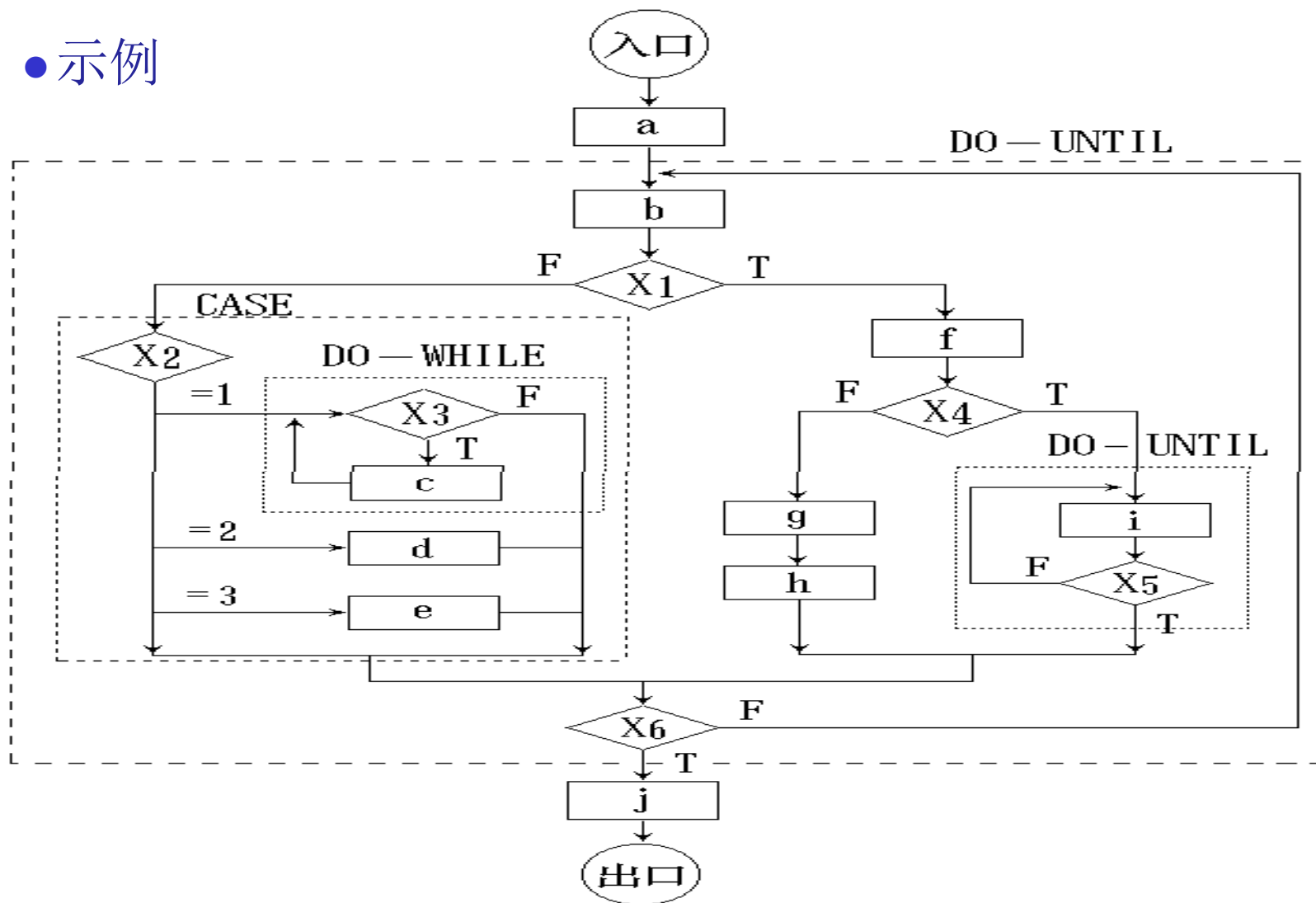
程序流程图基本结构

- 程序流程图使用五种基本控制结构如下图。
- 任何复杂的程序流程图都应由这五种基本控制结构组合或嵌套而成。



程序流程图--示例

● 示例



程序流程图的优缺点

- 优点：

对控制流程的描绘很直观，便于初学者掌握。

- 缺点：

(1) 本质上不具备逐步求精的特点，对于提高大型系统的可理解性作用甚微；

(2) 程序流程图中用箭头代表控制流，因此程序员不受何约束，可以完全不顾结构化程序设计的精神，**随意转移控制**；

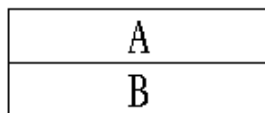
(3) 不易表示数据结构；

趋势：停止使用

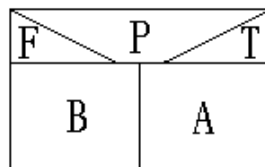
7.3.2 过程设计工具--N-S图

2. 过程设计工具：N-S图

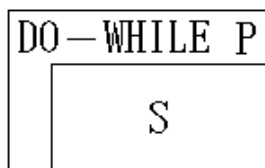
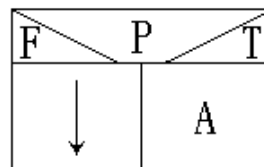
- N-S图是Nassi和Shneiderman提出来的，它体现了结构程序设计精神，是目前过程设计中广泛使用的一种图形工具。也叫盒图。



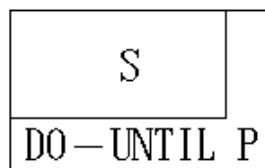
① 顺序型



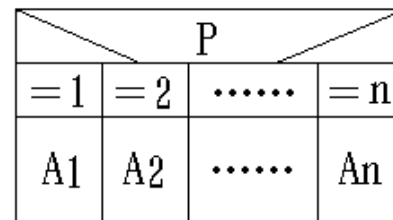
② 选择型



③ WHILE 重复型



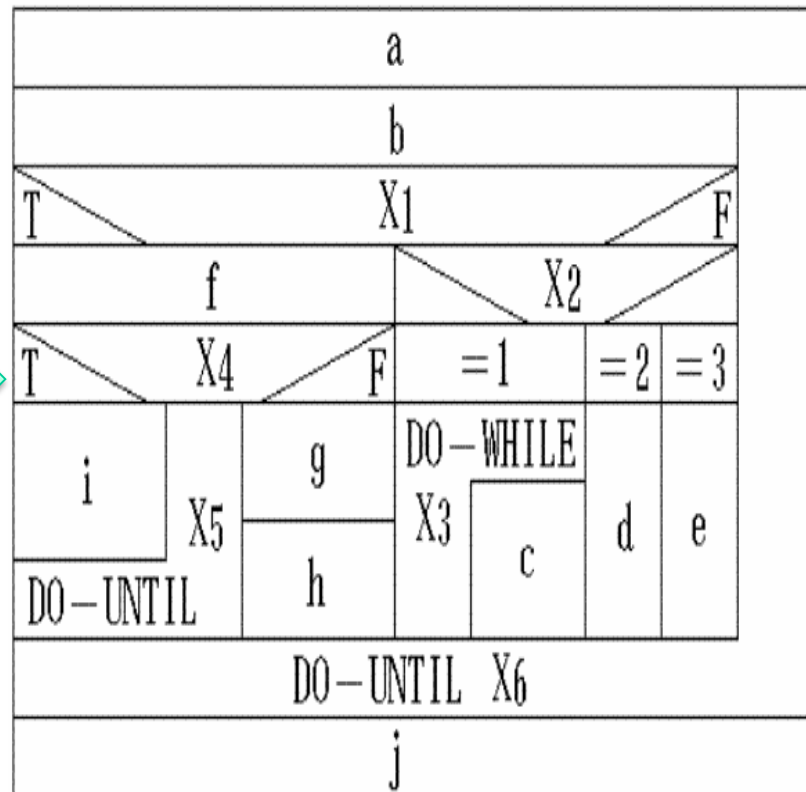
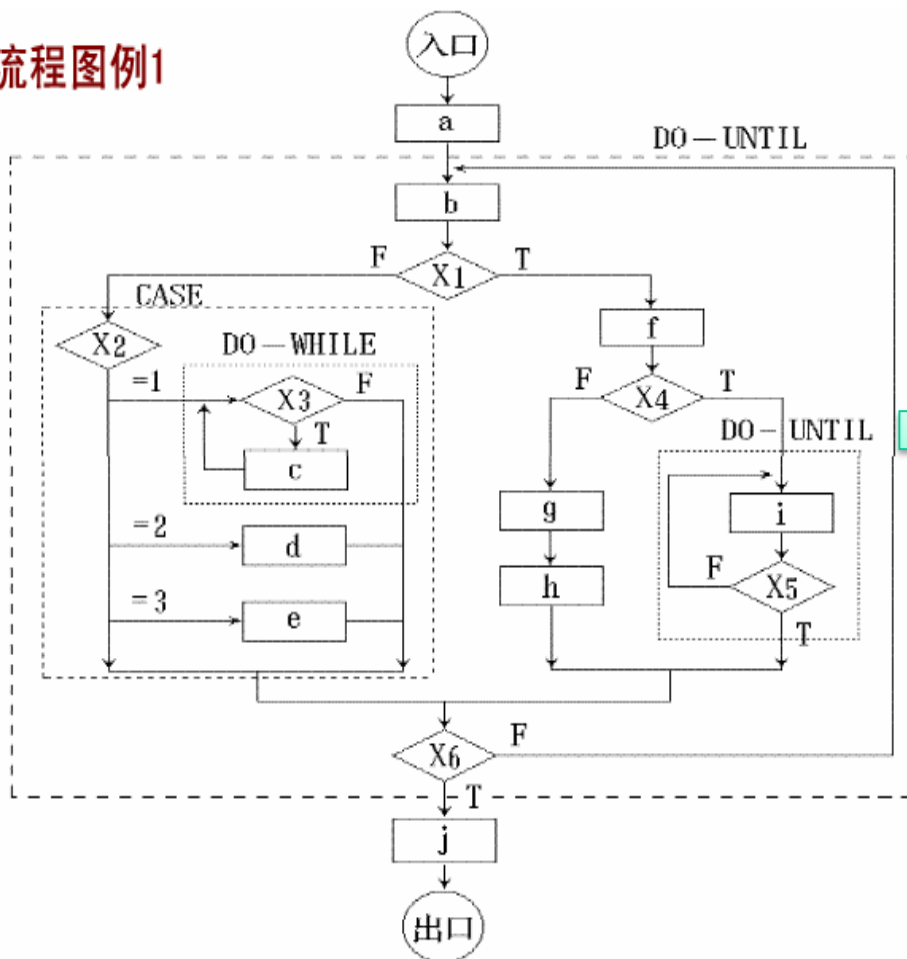
④ UNTIL 重复型



⑤ 多分支选择型
(CASE 型)

N-S图

流程图例1



N-S图的优缺点

● 优点:

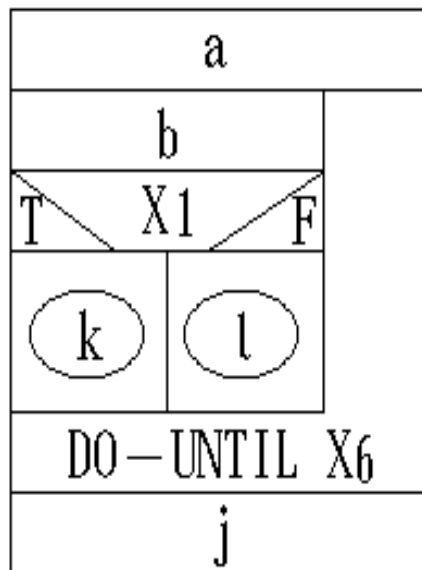
1. 功能域(一个特定控制结构的作用域)明确, 可以从盒图上一眼看出来。
2. 没有箭头不允许随意转移控制, **不可能任意转移控制**。
3. 很容易确定局部和全程数据的作用域。
4. 很容易表现嵌套关系, 也可以表示模块的层次结构。

● 缺点:

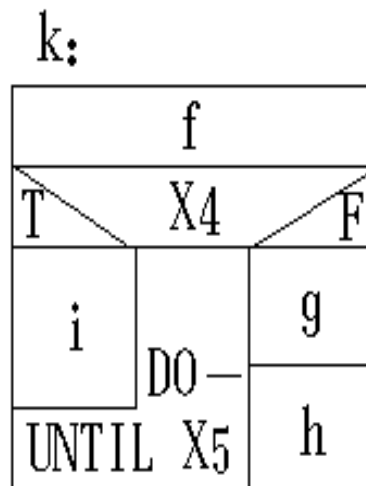
1. 随着程序内嵌套的层数增多时, 内层方框越来越小, 会增加画图的难度, 影响清晰度。

N-S图

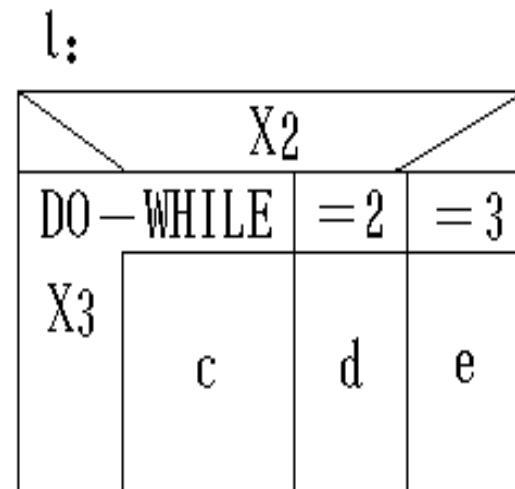
● N-S图的嵌套定义形式



(a)



(b)



(c)

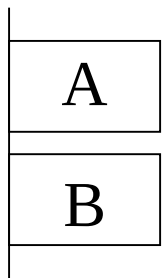
7.3.3 过程设计工具--PAD图

- PAD (Problem Analysis Diagram) 问题分析图, 日立公司中央研究所在1973年研究开发.
- 用结构化程序设计思想表现程序逻辑结构的图形工具, 现在已被ISO认可.
- PAD图是一种由左往右展开的二维树型结构。
- PAD图的控制流程为自上而下、从左到右地执行

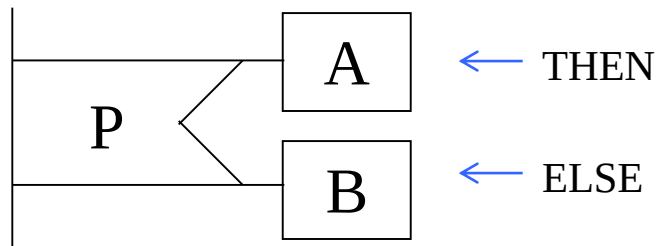
PAD的执行顺序

- PAD所描述程序的层次关系表现在纵线上。每条纵线表示了一个层次。
- PAD的执行顺序从最左主干线的上端的结点开始，自上而下依次执行。每遇到判断或循环，就自左而右进入下一层，从表示下一层的纵线上端开始执行，直到该纵线下端，再返回上一层的纵线的转入处。

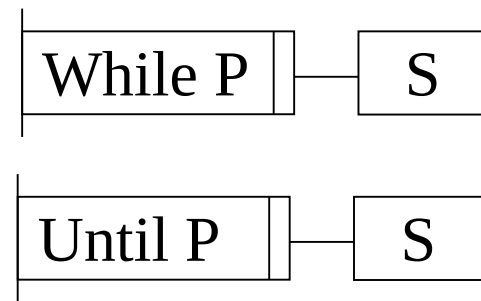
PAD图的结构



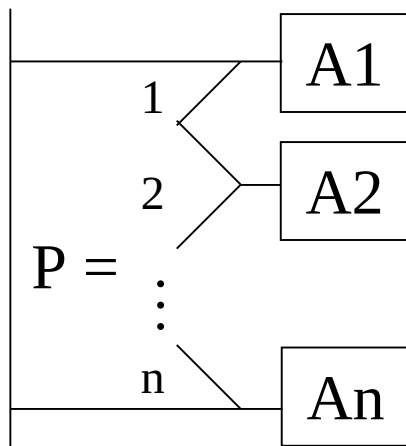
Sequential



Selective



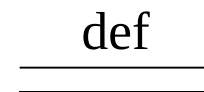
Loops



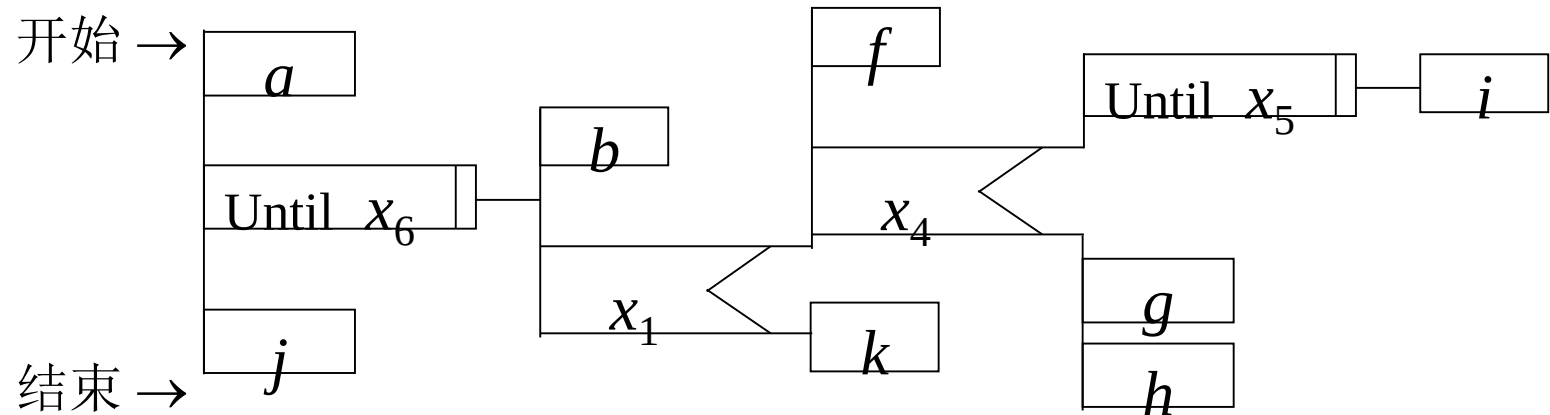
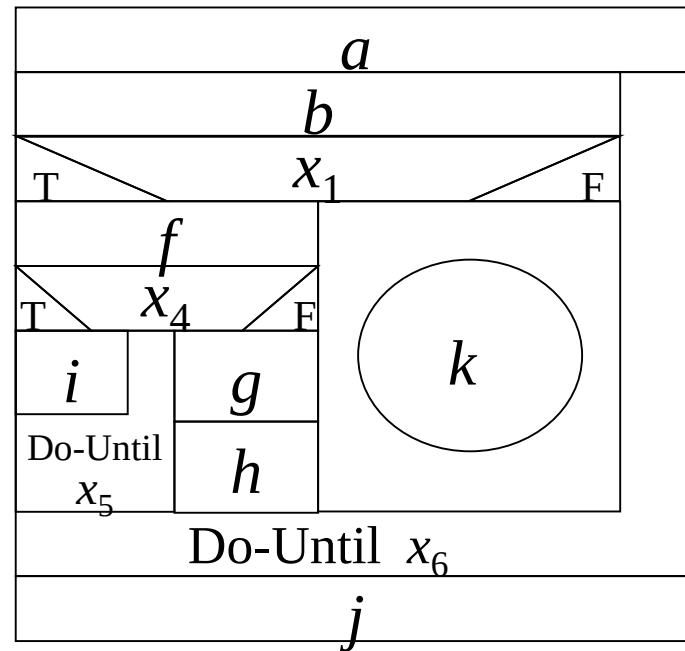
Case



Statement
Index

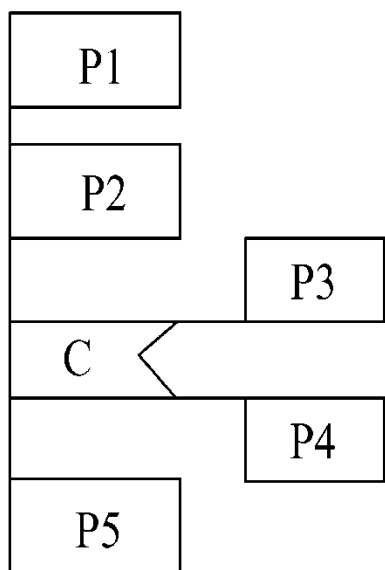


Definition

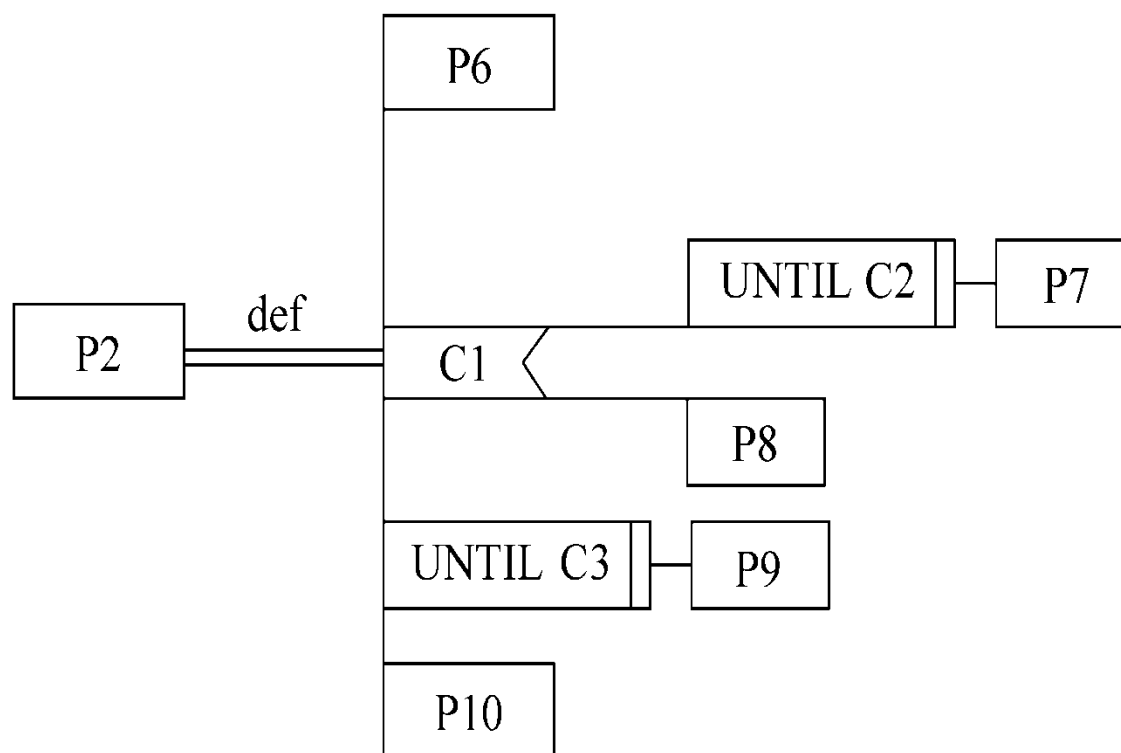


PAD图求精

- 例 使用PAD图提供的定义功能来逐步求精



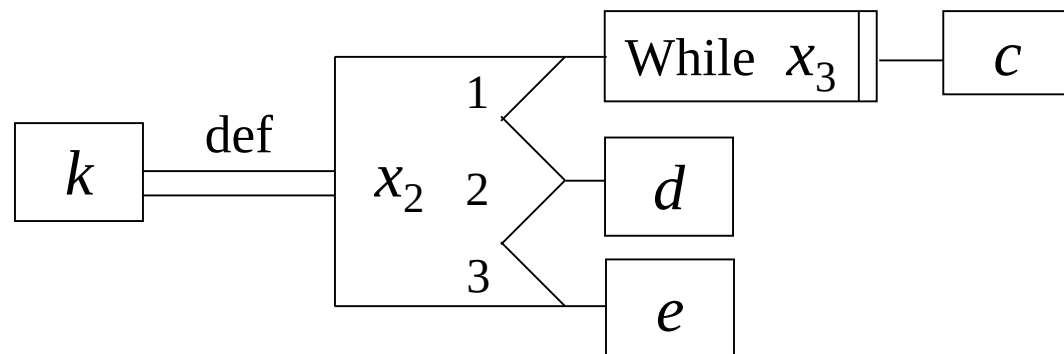
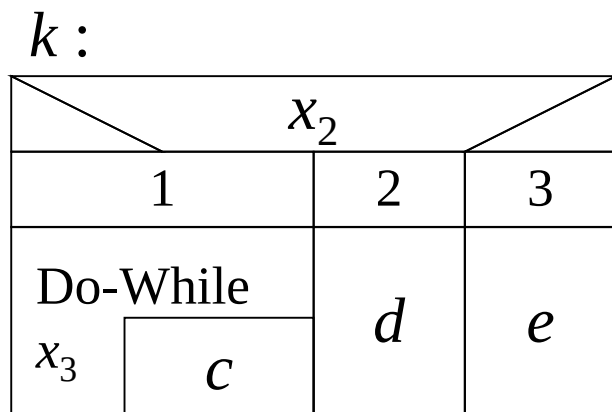
(a)



(b)

N-S图与PAD图的转换

- 使用PAD图提供的定义功能来逐步求精

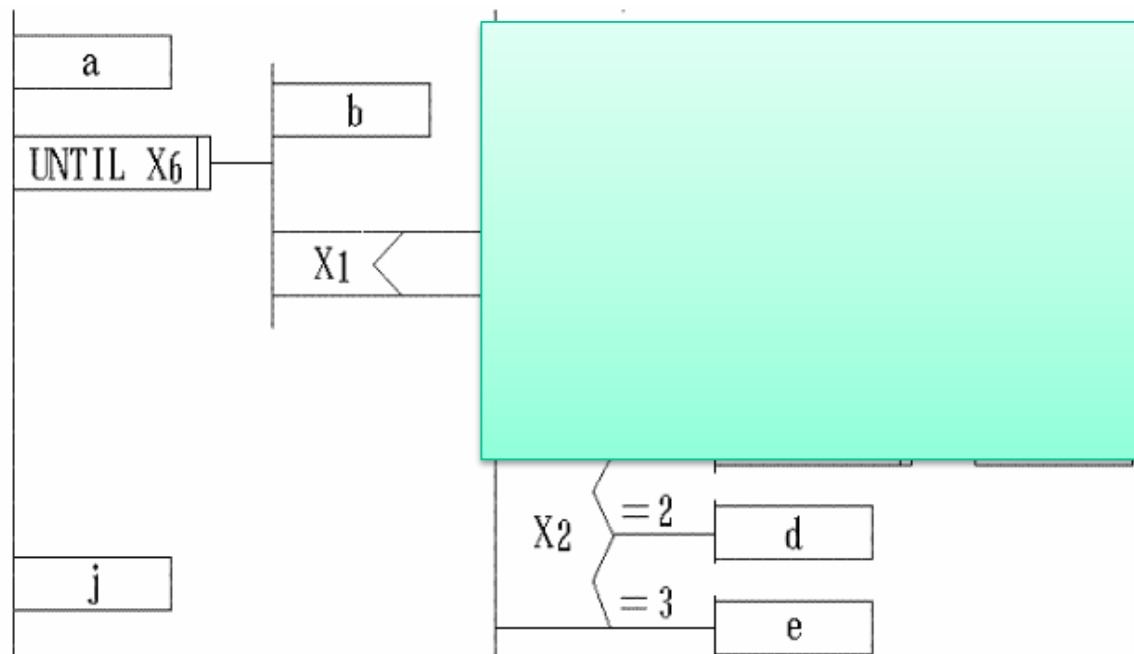
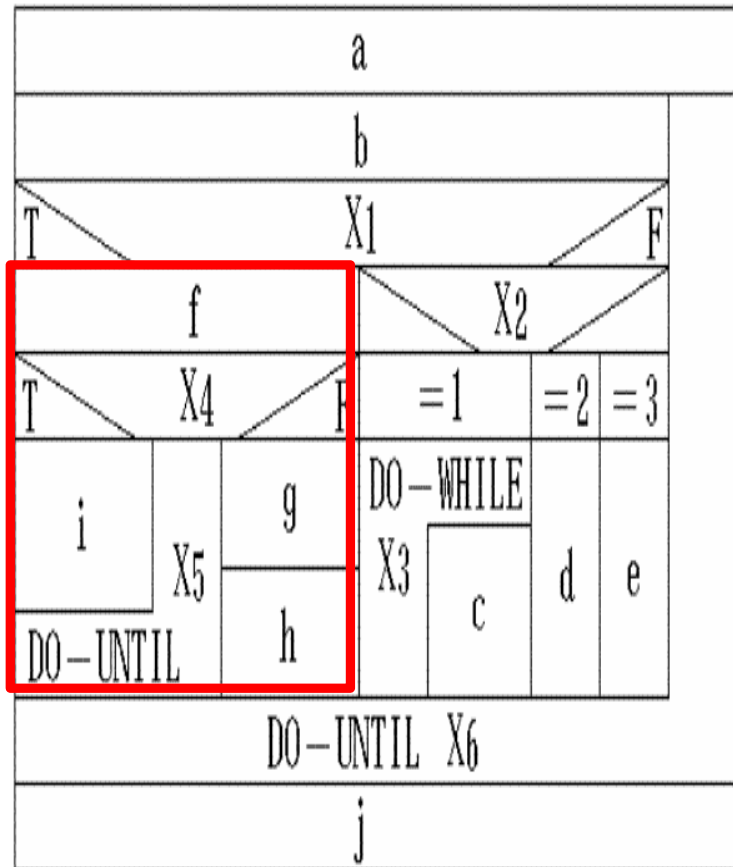


PAD图的优点

●PAD图的主要优点:

1. 程序结构清晰，结构化程度高。图中的竖线为程序的层次线，最左边竖线是程序的主线，其后再一层一层展开，层次关系一目了然。
2. 支持自顶向下，逐步求精的设计方法。
3. 既可以表示程序逻辑，也可以描绘数据结构。
4. 用PAD图表现程序逻辑，易读易写，使用方便。
5. 容易转换成高级语言源程序，也可用软件工具实现自动转换。

练习：下图中红框中转化为PAD图



过程设计工具—判定表

过程设计工具：判定表

- 判定表能够清晰地表示复杂的条件组合与所产生的动作之前的关系。判定表中条件取值只能是“T”和“F”。

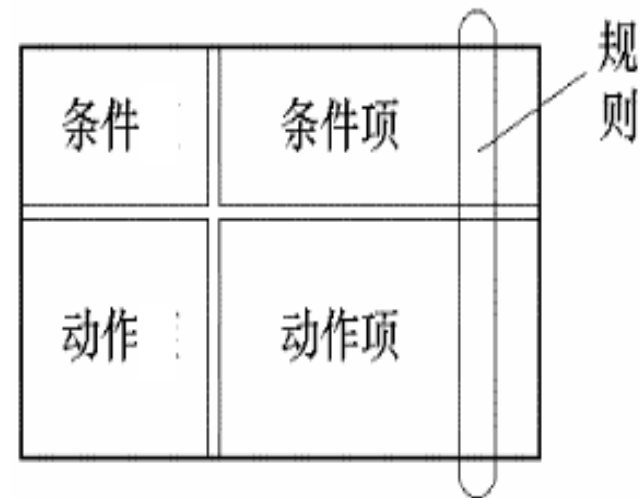
- 一张判定表由四部分组成：

左上部列出所有的条件 I

左下部是所有可能的操作 III

右上部是各种条件的组合矩阵 II

右下部是每种条件组合对应的动作



判定表举例

某航空公司规定，乘客可以免费托运重量不超过30公斤的行李。当行李重量超过30公斤时，头等舱国内乘客超重部分每公斤收费4元，其他舱的国内乘客超重部分每公斤收费6元，对于外国乘客超重部分每公斤收费比国内乘客多一倍，对于残疾乘客超重部分每公斤收费比正常乘客减少一半。

判定表举例

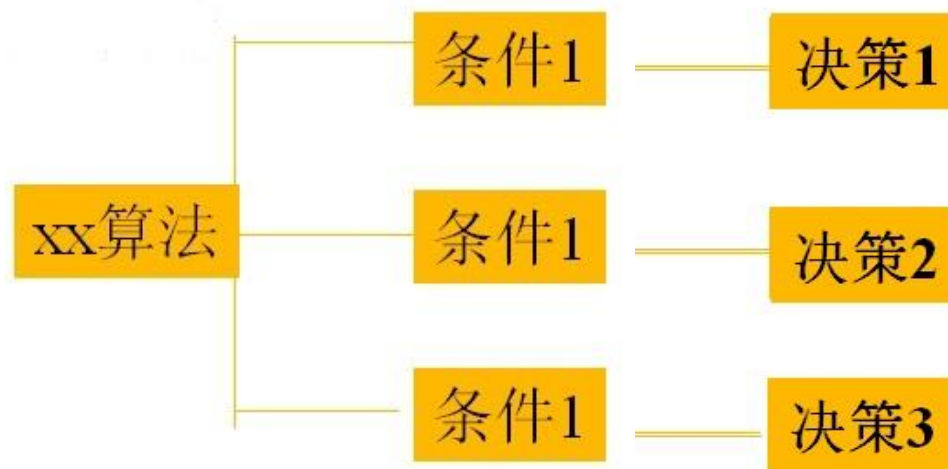
国内乘客		T	T	T	T	F	F	F	F
头等仓		T	F	T	F	T	F	T	F
残疾乘客		F	F	T	T	F	F	T	T
行李重量 ≤30	T	F	F	F	F	F	F	F	F
免费	√								
(W-30)*2				√					
(W-30)*3					√				
(W-30)*4		√						√	
(W-30)*6			√						√
(W-30)*8						√			
(W-30)*12							√		

判定表的特点

- 1. 当算法中包含多重嵌套的条件选择时，用程序流程图、N-S图或PAD都不易清楚地描述。然而，判定表却能清晰地表达复杂的条件组合与应做动作之间的对应关系
- 2. 判定表缺乏通用性，没法和PAD图那样良好的转换成代码
- 3. 无法清晰表示顺序和重复等处理

7.3.5 过程设计工具—判定树

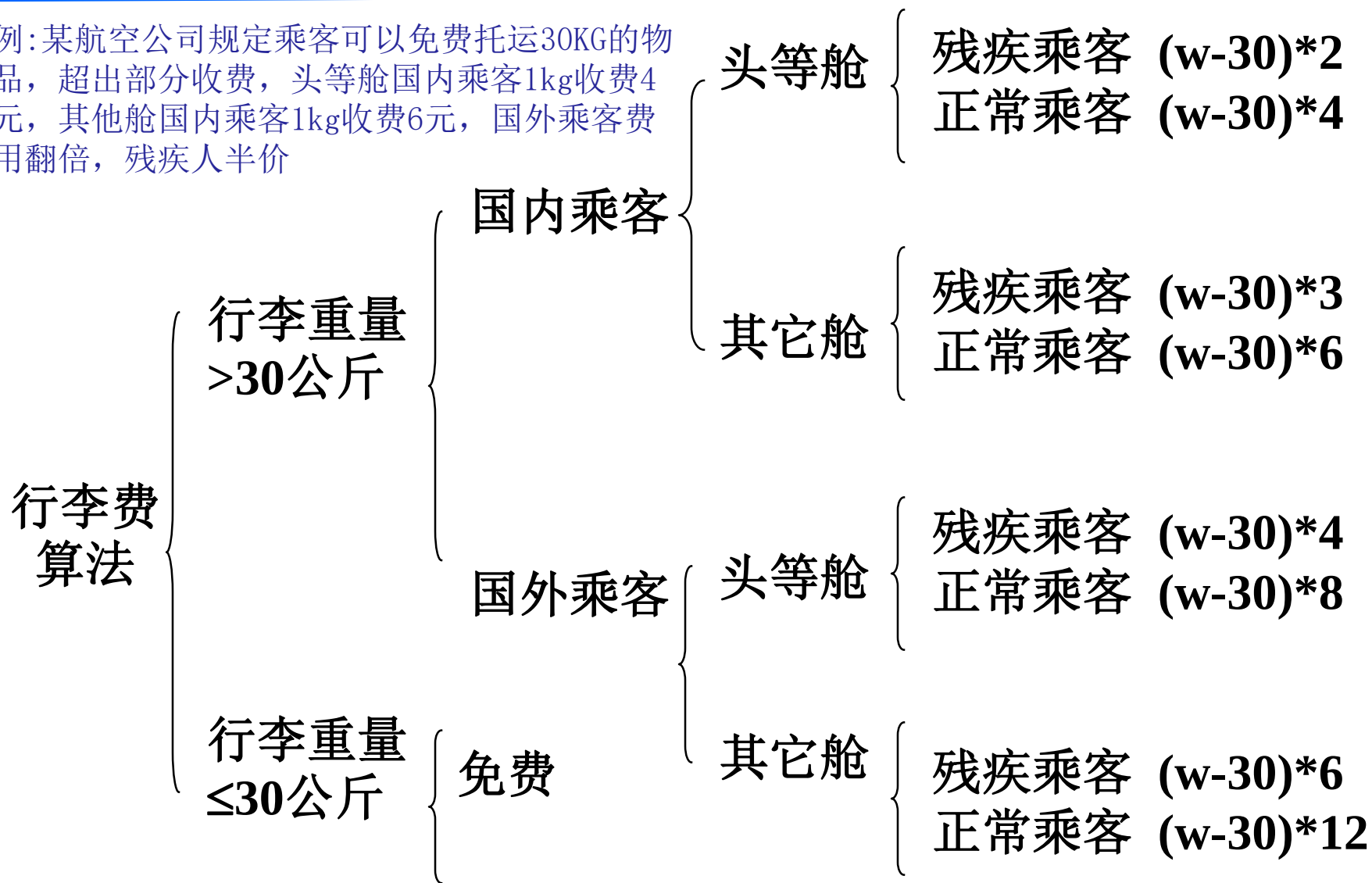
- 判定树又称决策树，和判定表类似，**适合描述问题处理中具有多重判断，即每个决策与若干条件有关。**
- 使用判定树进行描述时，应该从问题的文字描述中分清哪些是判定条件，哪些是判定的决策。
- 判定树的符号



判定树举例

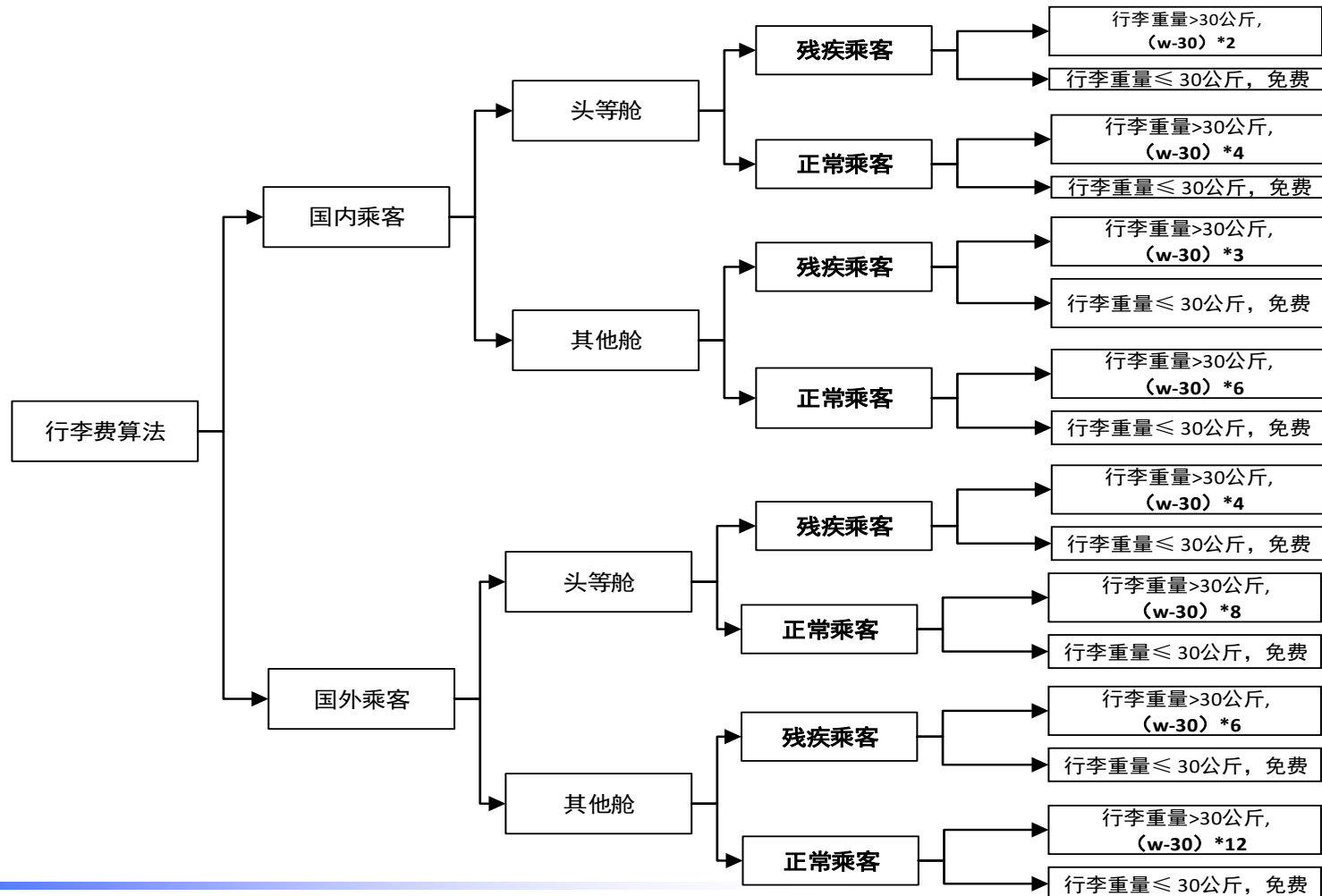
- 例：某航空公司规定乘客可以免费托运30KG的物品，超出部分收费，头等舱国内乘客1kg收费4元，其他舱国内乘客1kg收费6元，国外乘客费用翻倍，残疾人半价

例:某航空公司规定乘客可以免费托运30KG的物品,超出部分收费,头等舱国内乘客1kg收费4元,其他舱国内乘客1kg收费6元,国外乘客费用翻倍,残疾人半价



第一个分枝是至关重要的

判定树的分枝次序对于最终画出的判定树的简洁程度有较大影响，
所以**选择哪一个条件作为第一个分枝是至关重要的**。



判定树的特点

- 表示复杂的条件组合与应做的动作之间的对应关系。
- 判定树形式简单，长期以来一直受到重视。
- 判定树的简洁性不如判定表，经常出现同一个值重复写多遍，且叶端重复次数急剧增加。

7.3.6 过程设计工具--伪码和PDL

过程设计语言PDL (Program Design Language)

- PDL是一种用于描述功能模块的算法设计和加工细节的语言，称为设计程序用语言，也称伪码。
- 伪码的语法规则分为“外语法”和“内语法”。外语法应当符合一般程序设计语言常用语句的语法规则；而内语法可以用英语中一些简单的句子、短语和通用的数学符号，来描述程序应执行的功能。

PDL的一般语法规则

- 1) 在算法中的某些指令或子任务可以用文字来叙述，例如，“ x 是 A 中的最大项”，这样做的目的是为了减少与主要问题无关的细节，使算法本身更清晰。
- 2) 算术表达式可以使用通常的算术运算符，逻辑表达式可以使用关系运算符 $=$, $\neq$, $<$, $>$, $\leq$ 和 $\geq$ ，以及与（and）、或（or）、非（not）
- 3) 赋值语句是如下形式的语句： $a \leftarrow b$ ，语句的含义是将 b 的值赋给 a
- 4) 若 a 和 b 都是变量、数组项，那么记号 $a \leftrightarrow b$ 表示 a 和 b 的内容进行交换。
- 5) 条件语句有if-then格式和if-then-else两种形式：



PDL的一般语法规则

- 6) 有两种循环指令：while和for。
- 7) exit语句可以在通常的结束条件满足之前，被用来结束while循环或者for循环的执行。
- 8) return用来指出一个算法执行的终点。
- 9) 算法中的注释被括在/* */之中。

PDL举例：航空公司计算行李费

```
if (Weight>30) then
  if (Domestic passengers) then
    if (First class) then
      if (Passengers with disabilities) then
        cost=(Weight-30)*2
      else
        cost=(Weight -30)*4
      end
    else
      if (Passengers with disabilities) then
        cost=(Weight-30)*3
      else
        cost=(Weight -30)*6
      end
    end
  end
  .....
end
end
```

PDL的特点

- 有固定的关键字外语法，提供全部结构化控制结构、数据说明和模块特征。属于外语法的关键字是有限的词汇集，它们能对PDL正文进行结构分割，使之变得易于理解。
- 内语法使用自然语言来描述处理特性。内语法比较灵活，只要写清楚就可以，不必考虑语法错误，以利于人们可把主要精力放在描述算法的逻辑上。
- 可用于说明数据结构，包括简单的（如标量和数组）与复杂的（如链表和层次结构）的数据结构。
- 有子程序定义与调用机制，用以表达各种方式的接口说明。
- 使用PDL语言，可以做到逐步求精：从比较概括和抽象的PDL程序起，逐步写出更详细的、更精确的描述。